Docker Notes

Your Name

Date:

Contents

Chapter 1

Notes info introduction

Purpose of notes These are a set of notes on Docker.

Chapter 2

Geting Started with Docker

2.1 Base CLI Command

2.1.1 Docker Image commands

```
docker image build
docker image history
docker image inspect
docker image pull
```

2.1.2 Docker Container Commands

```
docker container create
docker container exec
docker container logs
docker container run
```

Can use man pages via CLI. See `docker container --help` See `docker container create --help`

Consider old syntax vs new syntax

```
docker run <image>
docker ps
docker logs <ps>
#--------------New syntex
docker container run
docker container ls
docker contiainer logs <id>
```

`docker ps` is the same as `docker container ls`

2.2 Images vs Containers

A Docker image is a standalone, read-only package that includes everything needed to run a software application: the code, runtime, system tools, libraries and settings.

- App Runtime

- Libraries
- Config and Env Vars
- Dependencies

Docker containers are running instances of images. See docker hub for more

Can launch multiple containers from a Single Docker Image Can launch multiple containers from different docker Images

2.2.1 Attached vs Detached modes

In most cases you will be running containers in detached modes, however there some specific use cases where you might wan tot run it in attached mode as well.

When you use the docker container run ¡image¿ without any extra flags, Docker starts the container in Attached Mode. In this mode, your terminal's local standard input, output and error streams are "attached" to the container.

Note: If you press Ctrl + C in ther terminal, you send a SIGINT signal to the container's primary process, which usually stops the container entirely.

To run a container in the background, you use the -d or –detach flag. This detached mode is the standard for any long-running service, for example web server containers or database containers. Ideally, you would like it to be runnig all of the time so you would use it with detached mode.

```
docker container run nginix
docker container run --detach nginix
docker container run -d nginix
```

Generally in most cases, you wil be using the detached mode in the production environments.

2.2.2 Basic Container Management Commands

When you have container running you can perform multiple operations on the container.

Managing the state of a container

```
docker container start <name>
docker container stop <name>
docker container restart <name>
```

Monitoring Performance

Example: A container can be consuming a certain level of memory and CPU

```
docker container stats <name>
```

This command can be useful in identifying which container is using the most CPU.

```
docker container inspect <name>
```

This command will get detailed information related to the container.

```
docker container rm <name>
docker container rm <name> --force
#--- use for exited containers
docker container ls -a
docker container rm <name1> <name2>
```

This will remove a container. If the container is running you can use force remove.

2.2.3 Basic Container Lifecycle

A docker container lives as long as its main process runs. If the process stops, the container stops. If the process is running the container will continue to run.

Short-lived containers - Use for scripts, batch jobs and one time commands.

Long Running Containers - Run a continuous process User for Web Servers, APIs, Database etc.

In production environments you will most likely use long-running containers.

Default Container Command

In Docker, the default container command is the instruction that tells the container process which process to run immediately upon starting. You can override the default container command if required.

2.2.4 Docker Container exec

`docker exec` lets you run a command inside an already running container.

```
docker container exec nginx ls -l
```

2.2.5 docker container exec - important of -i and -t flags

For simple commands that just print the output, you don't need any flags. The command runs, prints its result to your terminal and exits. No interaction is needed.

In most cases however you would want ot go inside the container, so you will require a shell. In such cases, you will require an additional set of flags that need to be added here.

If you want to connect inside a container using a shell like bash, sh (similar to SSH), the default approach will not work. Bash will close immediately. This is because it is an interactive shell that detects there is no input stream (STDIN) to read from. Without an active input stream or a terminal, it assumes its job is done and exists immediately.

2.2.6 The -i flag (interactive)

By default, Docker runs commands in non-interactive mode and does not keep the input stream (STDIN) open. using -i keeps STDIN attached to the container process, which is necessary for any interactive process.

```
docker container exec -i nginx bash
```

2.2.7 The -t Flag (TTY)

This will give you a full-fledged experience. Overview: the hyphen t flag allocates a pseudo terminal. If you just use the -t flag, the standard input get closed so that the keystrokes have no path to reach the process inside the container.

```
docker container exec -t nginx bash # This is very similar to SSH into a Linux
    server
```

Using the -it option, users can get a full interactive shell. You can type, navigate and explore.

```
docker container exec -it nginx bash
docker container exec -i -t nginx bash #Equivalent
exit # Return to the host system.
```

Using this command, you can run everything inside the container environment.

Note: Not every container comes with a shell, however the majority of them do. Note: Navigate Linux/OS file directory to see which binaries are available.

2.3 Basics of Ports

When data arrives at your computer or a server, the operating system needs to know which program should receive it. Ports help figure out which application should receive incoming data. An application can listen on a specific port number (port binding)

When incoming data comes, this data will also have the port number. The server will forward the data to the application with the matching port number. Ports are numbers from 0 to 65,535 and only one process can bind to a specific port on the same IP address at a time.

It is the responsibility of the incoming data to provide the port number so that the operating system can correctly map and send the data to the appropriate program over here.

```
netstat -ntlp # for linux. See what applications are bound to which ports.
```

2.4 Publishing Ports in Docker

When a container has been started, it is assigned a unique IP address within its configured Docker network. The application running inside the container may listen on a specific port.

Containers connected to the same network can communicate with each other using the contain's IP address and the application's listening port.

Note: The internal Docker network IP addresses are not directly accessible from the host system (In Windows and Mac OS). On linux hosts, the Docker bridge network is directly reachable from the host via container IP.

```
curl 172.17.0.2:80 # This will work on Linux not on windows or mac OS
```

2.4.1 Ports Concept

When we talk about ports in Docker, we are always talking about two different locations. - Host Port (outside) - The port you type into your browser (e.g. localhost:8080) - Container Port (inside) - The port the actual application is configured to use (e.g. 80)

2.4.2 Connecting to Container Application

To allow a container to accept incoming connections from the host or the outside world, you must bind (map) it to a host port. Once this is done the users can send a request to the IP address of the server followed by the port number.

```
docker container run -d --name nginx -p 8080:80 nginx
curl 172.17.0.2:8080 # this should work after the above command
```

2.5 Docker Restart Policy

by default, Docker containers will no start when they exit or when the docker daemon is restarted. Sometimes the server restarts and we want all the the containers to automatically be started again.

Docker provides restart policies to control whether your containers start automatically when they exit, or when Docker restarts.

2.5.1 Restart policies

You can specify the restart policy by using the –restart flag with the docker run command.

- no - The defaul. Docker will not restart the container under any circumstances
- on-failure - Restarts only if the container exits with a non-zero exit code
- unless-stopped - Always restarts except when you manually stop the container. If manually stopped, it won't restart even after a daemon restart.
- always - Always restarts regardless of exit code, including after a daemon restart — even if the container way manually stopped before the daemon went down.

In production environments we want the containers to start back up again should the server restart.

2.5.2 Choosing the right policy

Policy	Restarts on crash?	Restarts on success exit?	Restarts after daemon restart	
no	No	No	No	Deve
always	Yes	Yes	Yes	Critical se
on-failure	yes	no	Only if it was running	Bat
unless-stopped	Yes	Yes	Only if no manually stopped	Databases

Chapter 3

Creation, Management and Registry

3.1 Basics of a Dockerfile

A docker container is a running instance of image, whatever container you make is based of one of these docker images. There are numerous images available. From these images, multiple containers can be created.

For an organisation, you may want your own customized images.

A dockerfile is a simple text file containing ordered step-by-step instructions (commands) used to build a Docker image.

The workflow is, docker file into docker image into a running docker container. To create a docker file create a a file called ”Dockerfile”.

```
docker build -t image_name . # will reference the docker file in the same folder
docker container run -d -p 8080:80 --name container_name container_image:latest #
    Create the container
```

There is a wide range of instructions available that can be used in a Dockerfile to build and image.

Some of these include:

Instructions	Description
FROM	The starting point. Every Dockerfile must start with this
CMD	The default command that executes
RUN	Executes a command during the build process (e.g installing a package)
COPY	Copies files from your local machine into the container
SHELL	Set the default shell of an image

3.1.1 Format of a Dockerfile

The format of a Dockerfile follows the syntax INSTRUCTION ¡arguments¿

3.2 Docker file Instructions

Before you can createa a docker file the necessary set of instructions. See Docs for the set of instructions. Using this you can build a custom set of images.

3.2.1 FROM

The FROM instruction sets the base image that all subsequent instructions build upon.

3.2.2 RUN

The RUN instruction executes a command during the build process `RUN apt-get install nginx`

3.2.3 CMD

CMD instruction allows users to provide the default command that executes when the container starts.

3.2.4 Example DockerFile

Prior to building a docker image, try it in a base container.

```
docker container run -d --name ubuntu-container ubuntu:latest sleep 3600 # Run the
     container
docker container exec -it ubuntu-container bash # Open a bash shell on the
    container to check if desired programs are installed

# --------- from inside the container -----
apt update
apt install iputils-ping -y # Install ping for this example. -y so no prompt is
    asked.
```

Now we can map this to instructions in the DockerFile.

```
1 FROM ubuntu:latest
2
3 RUN apt update
4
5 RUN apt install iputils-ping -y
6
7 CMD ["ping", "google.com"]
```

Listing 3.1: Example Dockerfile

Using this image file we can build an image.

```
docker build -t <image-name> . # Ensure that you are in the right folder
docker container run --name <container-name> <container-image>
```

3.2.5 COPY and ADD

COPY and ADD serve the same fundamental purpose ”they can move files from you host machine into the container’s file system”

Think of COPY as a simple ”Ctrl+C, Ctrl+V.”. It is transparent and predictable. Docker official documentation recommends using COPY unless you have a specific reason not to.

Feature	COPY	ADD
Primary Function	Copies local files / folder to the image	Copies local files / folders to the image
Remote URLs	Not Supported	Can download files from a URL
Auto-Extraction	Does not extract compressed file	Automatically extracts .tar, gzip, etc
Best Practice	Recommended from most use cases	Use only when specific features are needed

3.2.6 ENTRYPOINT

Revising Basics of CMD Instruction

CMD allows users to provide the default command that executes when the container starts. CMD instructions can easily be over run by the docker run arguments.

ENTRYPOINT instruction

ENTRYPOINT sets the default executable for your container.

Any arguments supplied to the docker run command are appended to the ENTRYPOINT command

```
1  FROM ubuntu:latest
2
3  RUN apt update
4
5  RUN apt install iputils-ping -y
6
7  ENTRYPOINT ["ping"]
```

Listing 3.2: Example Dockerfile

```
docker container run --name ping-container-1 ping-image google.com # google.com is
    appended to the argument list of ENTRYPOINT
```

Notes: Use ENTRYPOINT when you need your container to always run the same base command and you want to allow users to append additional commands at the end. ENTRYPOINT can be overridden on the docker rtun command line by supplying the `--entrypoint` flag.

Command	Description
CMD	Defines the default executable of a Docker image. It can be overridden by argumetns
ENTRYPOINT	Defines the default executable. It can be overriden by the ”–entrypoint” docker run arguments

3.2.7 WORKDIR

WORKDIR sets the working directory inside the container for any subsequent instructions (RUN, CMD, ENTRYPOINT, COPY, ADD) Without WORKDIR, instructions like RUN, CMD, ENTRYPOINT, COPY and ADD execute from the container’s root directory (/)

```
1  FROM ubuntu:latest
2
3  COPY config.json /my-application #Specifying the folder is bad practice
4
```

```
5       ADD http://kplabs.in/friend.jpg /my-application
6
7       RUN cd /my-application && npm install
8
9       RUN cd /my-application && node server.js
```

Listing 3.3: Example Dockerfile

```
1       FROM ubuntu:latest
2
3       WORKDIR /my-application # Good practice
4
5       COPY config.json
6
7       ADD http://kplabs.in/friend.jpg
8
9       RUN npm install
10
11      CMD ["node", "server.js"]
```

Listing 3.4: Example Dockerfile

The WORKDIR instruction can be used multiple times in a Dockerfile.

3.2.8 EXPOSE

To connect to a container from the outside network, you must publish its port by mapping it to a port on the host machine (e.g. docker run -p 8080:80)

EXPOSE act as a documentation label that says ”This container listens on this port” It does not actually open any port to the outside world.

Note: If you use the -P flag when running a container (docker run -P), Docker will automatically map all EXPOSED ports to random high-order ports on your host machine.

Useful command

```
docker image inspect nginx-image
```

3.2.9 ARG

ARG instuction allows us to set build-time variables.

It allows you to pass specific values for a variable while it is being built.

```
1       FROM ubuntu:latest
2
3       ARG CONFIG_VERSION=1.2.0
4
5       RUN touch tmp-v${CONFIG_VERSION}.json /tmp
6
7       CMD ["sleep", "3600"]
```

Listing 3.5: Example Dockerfile

```
docker build --build-arg CONFIG_VERSION=1.2.5 -t test-image-02 . # Overrride the
    argument in the command line
```

The values of ARG do not persist once the image is built and the container is running.

3.2.10 ENV

The ENV instruction sets the environment variable inside a Docker image. These values will persist.

```
1  FROM ubuntu:latest
2
3  ENV PLANET="world"
4
5  CMD echo "Hello $PLANET"
```

Listing 3.6: Example Dockerfile

Setting and Overriding the Variables

You can use the -e, –env and –env-file flags to set simple environment variables in the container you're running, or overwrite variables that are defined in the Dockerfile of the image you're running.

```
docker container run --env PLANET=Pluto env-image-01
```

3.2.11 LABEL

LABEL adds key-value metadata to a Docker image.

Think of it like tags or sticky notes attached to your image — they don't affect how the container runs but provide useful information for humans and tooling.

```
1  FROM nginx:alpine
2
3  LABEL maintainer="maintainer maintainer@maintain.com"
4
5  LABEL use="prototype"
6
7  RUN echo '<h1>Hello</h1>' > /usr/share/nginx/html/index.html
8
9  CMD ["nginx", "-g", "daemon off;"]
```

Listing 3.7: Example Dockerfile

If you have hundreds of images on your server, you cna filter them by their labels.

```
docker images --filter "label=use=prototype"
```

3.3 Docker Image Prune

Unused docker images can quietly take up valuable disk space, leading to slower system perofrmance and potential storage issues. Regularly pruning them helps maintain a clean, efficient and prodiction ready environment.

```
docker image prune # Only removes dangling images - A dangling image is one that
    isn't tagged and isn't referenced by any container.
docker image prune # All imags not used by any container.
```

3.4 Layers of a Docker Image

A Docker image is not a single large file.

It's built as a stack of read-only layers, where each layer represents a discrete change.

Note: Key instructions like RUN, COPY and ADD create layers in your final image.

Once an image layer is built, it is read-only. It cannot be changed.

```
1  FROM ubuntu:latest
2
3  RUN apt update
4
5  RUN apt install nginx -y
6
7  RUN apt install nano -y
8
9  RUN echo '<h1>Hello from container</h1>' > /usr/share/nginx/html/index.html
10
11 CMD ["nginx", "-g", "daemon off;"]
```

Listing 3.8: Example Dockerfile

```
docker image history <image-name>
```

0B instruction are considered meta data.

The writeable layer: When a container runs, Docker adds one thin writable layer on top (the container layer). All changes made during the container's lifetime are stored in this thin R/W layer, leaving the underlying image layers untouched and immutable.

One Image and Multiple Containers: If multiple containers are launched from the same image, each container will have its own indpendent Read/Write layer. These containers will need to be removed if you wish to delete the image.

3.5 Docker Build Cache

Docker images are built layer by layer from a Dockerfile.

Each instruction that modifies the filesystem — such as RUN,COPY and add — adds a new read-only layer to the image stack.

Docker Build Cache reuses these layers from previous builds when the instruction (and its inputs) haven't changed. This can significantly improve the overall build time for the docker image and is more efficient.

Note: The build cache follows a chain: if a lyer is modified (e.g. a file change in a COPY instruction), every subsequent layer is invalidated and forced to rebuild.

3.6 Dockerfile - HEALTHCHECK instruction

HEALTHCHECK instruction - Docker allows us to tell the plaform how to test that our application is healthy. When Docker starts a container, it monitors the process that the container runs. If the process ends, the container exits.

That's just a basic check and does not necessarily tell the detail about the application.

```
1 FROM busybox
2
3 HEALTHCHECK --interval=5s CMD ping -c 1 172.17.0.2 #Health check runs every 5
      seconds.
```

Listing 3.9: Example Dockerfile

Parameters for HEALTHCHECK

We can specify certain options before the CMD operation, these include:

HEALTHCHECK –interval=5s CMD ping -c 1 172.17.0.2 –interval=DURATION (default: 30s) –timeout=DURATION (default: 30s) –start-period=DURATION(default: 0s) -retries=N(default: 3)

3.7 HEALTHCHECK Instruction Options

HEALTHCHECK instruction is combined with various options to get the desired results.

–interval=DURATION (default: 30s) –timeout=DURATION (default: 30s) –start-period=DURATION(default: 0s) -retries=N(default: 3)

If you do not specify these options while adding a health check instruction, then the associate default values will automatically be used.

Exist Status

The command's exit status indicates the health status of the container The possible values are: 0: Success — The container is healthy and ready for use 1: Failure — The container is not working correctly 2: Reserved — Do not use this exit code

Example use-case

Check every five minutes or so that a web-server is able to serve the site's main page within three seconds:

```
1 HEALTHCHECK --interval=5m --timeout=3s \ CMD curl -f http://localhost/ || exit 1
```

Listing 3.10: Example Dockerfile

Health checks can also be performed using the docker health check command. SEE DOCS

```
docker run -dt --name tmp --health-cmd "curl -f http://localhost" --health-
    interval=5s busybox sh
```

Overview of CURL

CURL is used with -f option to fail silently. This is mostly done to better enable scripts etc to better deal with failed attempts

3.8 Docker commit

Whenever you make changes inside the container, it can be useful to commit a container's file chagnes or settings into a new image. By default, the container being committed and its processes will be paused while the image is comitted.

```
docker container commit CONTAINER-ID myimage01 # Syntax
```

Change Option while Commiting

The –change option will apply Dockerfile instructions to the image that is created.

Supported Dockerfile instructions: - CMD— ENTRYPOINT — ENV — EXPOSE — LABEL — ONBUILD — USER — VOLUME — WORKDIR

Whenever you make changes inside the container, it can be useful to commit a container's file change or settings into a new image. By default, the container being comitted and its processes will be paused while the image is comitted.

```
docker container commit CONTAINER-ID myimage01
```

3.9 Flattening Docker Images

Generally, the more layers and image has the more the size of the image. Some of the image size goes from 5GB to 10GB. Flattening an image to a single layer can help reduce the overall size of the image.

```
docker image history <image> # Get image size
docker export myubuntu > myubuntudemo.tar # Export file
cat myubuntu.tar | docker import - myubuntu:latest
docker images # Verify the image is there
docker image history ubuntu # Should see that the image has only one layer and
    reduced size
```

This is referred to as, flattening the image.

3.10 Overview of Docker Registries

A registry is similar to a github. Generally whenever you code for system administrator, it might be some kind of confgiruation management scripts like Ansible etc. Whenever you write those scripts it is recommedned to put it in a centralized location where other people can access as well. The code will remain safe there because if you store it locally and the machine fails then your code will be lost. This is the reason you store it in a centralized location.

The same is true for the centralized docker registry.

3.10.1 Understanding Docker Registry

A Registry is a stateless, highly scalable server side application that store and lets you distribute Docker images. Docker Hub is the simplest example. There are various types of registry available, which includes:

- Docker Registry
- Docker Trusted Registry
- Private Registry (AWS ECR)
- Docker Hub

```
docker tag ubuntu:latest localhost:5000/myubuntu
docker images # check if the image is there
docker push localhost:5000/myubuntu # This will push to a docker registry
#----------- pulling an image from the repository
docker pull localhost:5000/myubuntu
docker image # Verify that the image has been pulled from the registry.
```

3.11 Pushing Images to a Central Repository

```
docker login # Login into docker hub
docker tag busybox 5066/mydemo@v1
docker push 5066/mydemo
#----- verify in docker hub
docker pull 5066/mydemo:v1 # pull image from docker hub
docker image # verify
docker logout # log out of docker hub
```

The same goes for other private repositories like ECT. If you want to push certain images to ECR or if you want to pull certain images from ECR then you will have to login. Only after then will you be able to pull or push the images.

Specifically this is related to the private repositories.

3.12 Applying Filter for Docker Images

Searching for docker images in the docker registry.

SEE DOCS also

```
docker search nginx
docker search nginx --limit 5 # Will list the top five starred repos
docker search nginx --fitler "is-official=true"
```

3.13 Moving Images Across Hosts

Overview of the Docker save command: The docker save command will save one or more images to a tar archive

```
docker save busybox > busybox.tar
```

Overview of Docker Load: The docker load command will load an image from a tar archive

```
docker load < busybox.tar
```

Chapter 4

Networking

4.1 Overview of Docker Networking

Some docker containers may want to communicate with the internet, some might want to be in the isolated conditions. Specifically the containers which might have critical data.

Docker takes care of the networking aspect so that the container can communicate with other containers and also with Docker Host.

4.1.1 Overview of Network Drivers

Docker networking subsystem is pluggable using drivers. There are several drivers available by default and this provides core networking functionality.

- Bridge
- Host
- Overlay
- Macvlan
- none

If you are using Docker community edition, by default you will have bridge, you will have host and you will have none. If you are using Docker Enterprise, you will also have the overlay one.

```
docker network ls # see associated networks
docker inspect bridge # See more details about the bridge network
```

```
docker container run -dt --name myhost --network host ubuntu # specify a network
    for the container to run in.
docker network inspect host # Check the container is running in the network
```

4.2 Understanding Bridge Networks

A bridge network basically makes uses a software brdige which allows containers connected to the same bridge network to communicate, while providing isolation from containers which are not connected to that bridge network.

```
docker container inspect demo # See the network that a container is associated
    with
```

The containers which are present under the bridge network will be able to commnicate with each other. These containers have the interface.

Note: Whenever you create a container and you don't specify the network, it gets creted on the bridge network.

This automatic communication between containers is for the bridge network.

4.2.1 Overview of Network Drivers

Bridge is the default network driver for Docker. If we do not specify a driver, this is the type of network you are creating. When you start Docker, a default bridge network (also called a bridge) is created automatically and newly-started containers connect to unless specified.

We can also create User-Defined Bridge Nework which is superior to the default bridge.

4.3 Implementing User-Defined Bridge Networks

4.3.1 Overview of User-Defined Bridge

There are various differences between default and user-defined bridge. Some of these include:

- User-definedd bridges provide better isolation and interoperability betwen containerize applications
- User-defined bridge provide automatica DNS resolution betwen containers
- Containers can be attached and detached from user-defined networks on the fly.
- Each user-definedd network creates a configurable brdige.
- Linked containers on the default bridge network share environments.

In order to create a container in the user defined bridge, you will first need to create the user defined bridge. You can give any name to the network, however the driver which is connected to the network matters.

```
docker network create --driver bridge mybridge
docker network ls # Should see the above network in the list
docker network create mybridge-demo # No specifed driver. Docker will default to
    the bridge driver.
```

4.4 Understanding Host Network

4.4.1 Overview of Host Network

This driver removes the network isolation between the docker host and the docker containers to use the host's networking directly. For instance, if you run a container which binds to port 80 and

you use host networking, the container's application will be available on port 80 on the host's IP address.

Example: Want to deploy a service like an intrusion detection system. You cannot install that in a container with a bridge network because if you do that, then the IDDs will monitor the traffic of the container. It will not monitor that of the host.

However if you want to monitor in such a way that containers application can monitor the traffic of eth0 of the host, you need make use of the host network.

4.5 Implementing None Network

4.5.1 Overview of None Network

If you want to completly disable the networking stack on a container, you can use the none network. This mode will not configure any IP for the container and doesn't have any access to the external network as well as for other containers.

The none network is basically launch in an isolated network stack.

4.6 Publish All Argument for Exposed Ports

4.6.1 Overview of Port Publishing

```
docker container run -dt --name webserver -p 80:80 nginx
```

There is also a second approach to publishing all the exposed ports of the container.

```
docker container run -dt --name webserver -P nginx
```

This is also referred to as a publish all . In this approach, all exposed ports are published to random port of the host.

Example command:

```
docker contaienr run -dt -P --name webserver02 nginx
```

So generally whenever you make use of -P option, what happens is that any port which is exposed in a specific image the hyphen P option will correspondingly create a random port in the host and it will make it the exposed port of the container. In this case you don't really have to explicitly specify the port mapping. It automatically does it for you.

4.7 Legacy Approach for Linking Containers

This is similar to one of the features which have been provided by the customer defined bridge network.

```
docker container run -dt --name container01 busybox sh
docker contaienr run -dt --link container01:container --name container02 busybox
    sh
```

This is in a default bridge network. This is not the user defined bridge network.

The –link option was something which is legacy. It is the older approach of defining things.

Later, docker came up with an approach where this kind of feature is already available with the user defined bridge network. It is not reccomended to be used now.

Chapter 5

Orchestration

5.1 Overview of Container Orchestration

During the initial times, users relied on manual container management or basic scripts to handle tasks like deployment, scaling and monitoring. When the environment becomes larger both the approaces are not really sufficient to handle things at scale.

5.1.1 Challenge 1 - Manual Scaling

If an application needed to handle higher traffic, developers had to manually start additional containers.

This involved identifying which servers had enough resources, deploying new containers etc.

5.1.2 Challenge 2 - Lack of Fault Tolerance

If a container failed (e.g. due to a crash or resource exhaustion), it wouldn't automatically restart withouth manual intervention in most of the cases.

5.1.3 Introducting container orchestration

Container orchestration automates the deployment, management scaling and networking of containers. In the newer approach where a container orchestration tool is used, the user will communicate directly with the container orchestration tool. And the container orchestration tool will in turn take care of the responsibility or launching the containers in the appropriate server, making sure that the containers are always running.

If the containers are not running, say in the event of a crash, then the contaienr orchestration tool has the logic to migrate the container or relaunch the container in another server to ensure that containers are always running.

Most common tasks related to deployment, scaling, networking etc is taken care of by the container orchestration tool.

Note: Orchestration tools can automatically scale containers up or down based on defined polices and real-time traffic. Traffic will be routed equally amongst all the servers.

Failed containers are automatically restarted or replaced to ensure high availability. The container orchestrator has the appropriate lofic to create on more set of containers in another server and redirect traffic to the server.

```
docker server create --replica=5 --name webserver nginx
```

5.1.4 Additional things provided by container orchestration tool

Container orchestration is the process of automating the networking and management of containers so you can deploy applications at scale.

- Provisioning and deployment
- Configuration and scheduling
- Resource allocation
- Load balancing and traffic routing
- Monitoring of container health

5.1.5 Container orchestration tools

There are multiple set of Contaier Orchestration tools available in the industry
Based on the organisation and requirement, yyou cna choose tools per preference.

- Docker Swarm - Simpler setup, less expertise required
- Kubernetes - Enterprise scale
- Amazon ECS - Has a GUI, cloud based

The main purpose of the container orchestration tool remains the same, however different tools are a different set of features. The difference in features can be drastically different.
In large scale organisations with manpower and expertise where there is a requirement for 1000s of containers Kubernetes is a good choice.

5.2 Overview of Docker Swarm

Docker Swarm is a container orchestration tool which is natively supported by Docker.

5.2.1 Useful bash script

Script for installing docker on nodes/virtual machines.

```
#!/bin/bash
yum-config-manager --add-repo https://download.docker.com/linux/centos/docker-ce.
    repo
yum -y install docker-ce
systemctl start docker
systemctl enable docker
```

5.3 Initializing Docker Swarm

- A *A node is an instance of the Docker engine participating in the swarm*
- To deploy your application to a swarm, you submit a service definition to a manager node.
- The manager node dispatches units of work called tasks to worker nodes.

```
# Run in the respective node
docker swarm init --advertise-addr <MANAGER-IP> # Mangager Node Command
docker swarm join-token worker # Worker nodes command

docker swarm init --advertise-addr <eth0 ip obtained via ifconfig> # Allows worker
     node to joing the swarm
```

5.4 Services tasks and containers

A service is the definition of the tasks to execute on the manager or worker nodes.

```
docker service create --name webserver --replicas 1 nginx #This command will
    create a container in one of the nodes which are a part of the swarm
```

5.4.1 Services, Tasks and Containers diagram

5.5 Scaling Services in Swarm

Once you have deployed a service to a swarm, you are ready to use the Docker CLI to scale the number of containers in the service. Containers running in a service are call ”tasks”

```
docker service scale webserver=5
docker service ps webserver # This should show five tasks
docker service scale webserver=1 # Scale down. Verify with above command
```

5.6 Multiple Approaches to Scale Swarm Services

There are two ways you can scale the services in swarm.

- *docker service scale webserver=5*
- *docker service update –replicas 5 webserver*

The differences between the commands. Within the scale command we specify multiple services. `docker service scale service01=3 service02=3` The service command only accepts one service.

5.7 Replicated vs Global Service

There are two types of service deployments, replicated and global. For a replicated service, you specify the number of identical tasks you want to run. Example, decide to deploy an NGINX service with two replicas, each serving the same content.

5.7.1 Global Service

A global serivce is a service that runs on task on every node. Each time you add a node to the swarm, the orchestrator creates a task and the scheduler assigns the new task to the new node.

```
docker service create --name antivirus --mode global -dt ubuntu # This will launch
    the task antivirus in each and every node.
```

5.8 Draining Swarm Node

Consider that you want to bring down a container/server to do patches or maintenance etc. In that case it is reccomended that you drain the specify node so that whatever containers that the swarm might be running in this node would be switched to a different availabel node.

It is not really recommended to directly go ahead and stop the Docker daemon or restart.

You can migrate all the contaienrs which are running to a active node which you know is going to run for a longer period of time and that can be acheived with the help of draining.

```
docker node update --availability drain swarm03 # drain node with name swarm03
docker node ls # Veritify that swarm03 availbility is now drain
docker node update --availability active swarm03 # drain node with name swarm03
```

5.9 Inspecting Swarm Service and Nodes

Docker inspect provides detailed information of the Docker objects. This includes:

- Docker Containers
- Docker Network
- Docker Volumes
- Docker Swarm Services
- Docker Swarm Nodes

```
docker service inspect demotroubleshoot --pretty # JSON withouth pretty flag
docker node inspect <ID> --pretty
```

5.10 Adding Network and Publishing Ports to Swarm and tasks

Generally whenever we start a container on a bridge network, we specify the port mapping from the host to the container.

```
docker service create --name mywebserver --replicas 2 --publish 8080:80
```

5.11 Overview of Docker Compose

The Docker compose tool is basically a tool used for defining and running multi-container Docker applications. In production an application my require two containers or three containers. In order to achieve that correcly docker compose is a tool you may use.

With compose you use a YAML file to configure your applications services.

We can start all the services with a single command `docker compose up` We can stop all the services with a single command `docker compose down`

Docker compose comes with the default docker package. In Linux you will have to install docker compose.

Listing 5.1: Configuration file

```
version: '3'
services:
    webserver:
        image: nginx
        ports:
            - "8080:80"
    database:
        image: redis
```

Use `docker-compose config` to verify the format is correct. Use `docker-compose up -d` To run this in detached mode.

5.12 Deploying Multi-Service Application in Swarm

With docker compose we can go ahead and deploy the containers within the same host. However within the swarm cluster the approach is a little different.

A specific web-application might have multiple containers that are required as part of the build process. Whenever we make use of docker service, it is typically for a single container image. The docker stack can be used to manage a multi-service application.

The docker stack command can take the docker compose YAML file and deploy it in the swarm environment.

5.12.1 Overview of Docker Stack

A stack is a group of interrelated services that share dependencie and can be orchestrated and scaled together. A stack can compose YAML?? file similary to Docker Compose YAML files. We can define everything within the YAML file that we might define while creating a Docker Service.

`docker stack deploy` is something that allows us to deploy the services across the swarm.

```
docker stack deploy --compose-file docker-compose.yml demo
docker stack ps demo # verify
docker stack rm demo
```

5.13 Locking Swarm Cluster

A swarm cluster can contain a lot of sensitive information.

- TLS key used to encrypt communication among swarm nodes
- Keys used to encrypt and decypt the Raft logs on disk

If your Swarm is compromised and if data is stored in plain-text, an attack can get all the sensitive information.

Docker Lock allows us to have contorl over the keys.

```
docker swarm update --help # Man pages
```

Autolock is one of the features. When it is set to true, the next time when you restart your swarm, the swarm will basically aks you for a password whenever you want to perform some kind of operation.

```
docker swarm update --autolock=true # Save the resulting key as it will be
    important
# restart docker container
sytemctl restart docker
docker swarm unlock
#> will be prompted for the key
docker swarm unlock-key # This will allow you to retrieve the key
# Rotating the key
docker swarp unlock-key # Use some kind of password manager
```

5.14 Troubleshooting Swarm Service Deployments

A service may be configured in such a way that no node currently in the swarm can run its tasks. In this case, the service remains in state pending. There are multiple reasons why service might go into pending state.

Examples when service go into the Pending State If all nodes are drained and you create a service, it is pending until a node becomes available.

You can reserve a specific amount of memory for a service. If no node in the swarm has the required amount of memeory, the service remains in a pending state until a node is available which can run its tasks.

You have imposed some kind of placement contstraints.

5.15 Mounting Volumes via Swarm

```
docker service create --name myservice --mount type=volume,source=myvolume, target
    =/mypath nginx
docker volume ls # verify that the volume has been created.
```

Volumes persist, even if you remove the container and you will still be able to access the data.

5.16 Control Service Placement

Docker Swarm has the capabilty to push the task or containers on the worker nodes depending upon the specific constraints tha twe can specify.

5.16.1 Overview of Placement Constraints

Swarm services provide a few different ways for yo uto control the scale and placement of service on different nodes.

- Replicated and Global Services
- Resource Constraints (requirements of CPU and Memory)

- Place Constraints (Only run on nodes with label pci$_{c}ompliance = true)Placement Preferences$

  Docker allows us to label each and every node, and depending on the lable we can have a placement preference.

```
docker service create --name myconstraint --constraint node.labels.region=blr --
    replicas 3 nginx
docker node inspect # You can see the labels here

docker node update --label-add region=uk <NODE_ID>
```

5.17 Overview of Overlay networks

The overlay network driver creates a distributed network among multiple Docker daemon hosts.

Overlay networks allow containers connected to it to communicate securely.

If a webserver wants to communicate with a webserver in another node; then this is not possible with the host network. In order for this communication to happen we need to make use fo the overaly network.

Whenever you initiate a swarm, youi will find that the overlay network has been initailized.

```
docker network create --driver overlay mynetwork
docker network ls # verify the overlay network has been created.
docker service create --name myoverlay --network mynetwork --replicas 3 nginx
docker node ls #See the nconnected nodes
```

5.18 Secure Overlay Networks

5.18.1 Overview of Secure Overlay Networks

For the overlay networks, the contaienrs can be spread across multiple servers. If the contains are communicating with each otehr, it is recommended to secure the communication. To enable encryption, when you create an overlay network pass the –opt encrypted flag

```
docker network create --opt encrypted --driver overlay my-overlay-secure-network
```

5.18.2 Important Notes

When you enable overlay encryption, Docker cretae IPSEC tunnels between all the nodes where tasks are schedueld for service attached to the overlay network. These tunnels also use the AES algorithm in GCM mode and manager nodes automatically rotate the keys every 12 hours. Overlay network encryption is no suport on Windows. If a Windows node attempts to connect to an encrypted overlay network, no error is deteceted but the node will not be able to commnicate.

5.19 Creating Swarm Services using Templates

We can make use of templates while running the service create command in swarm.

5.19.1 Example

```
docker service create --name demoservice --hostname="{{.Node.Hostname}-{.Service.
    Name}}" nginx
```

5.19.2 Valid Placeholders table

Placeholder	Description

5.19.3 Supported Flags

There are three supported flags for the placeholder templates:

- –hostname
- –mount
- –env

5.20 Split Brain and The Importance of Quorum

Consider two nodes with a running webservers. One is master and one is slave. Master is connected to the database. If for some reason the two cannot communicate the worker will be promoted to master, even though the origonal master is still running. Both will try and write to the database. This will could lead to corruption.

5.20.1 Quorum

A quorum is nothing by how many servers you have within a cluster. Having the correct quorum will help prvent you form having the split brain problem. In the quorum type of architecture, voting is extremly important.

5.20.2 Important notes for a Quorum

Cluster Size	Majority	Fault Tolerance

5.21 High Availability of Swarm Manager Nodes

Manager Nodes are responsible for handling the cluster management tasks. It is important that you mhave multiple manager nodes for high availability.

5.21.1 Manager Nodes

Manager nodes have many responsibilities within the swarm, these include:

- maintaining cluster state
- scheduling services
- serving swarm mode HTTP API endpoints

Using a Raft implementation, uthe managers maintian a consistent internal state of the entire swarm and all the services running on it.

Swarm comes with it's own fault tolerance features. Docker recommends you implemenmt and odd number of nodes according to your organisations high-availability requirements. Using a Raft implementation, the managers maintian a consistent internal state of the entire swarm and all the services runnign on it.

An N manager cluster trolerate the loss of at most (N-1)/2 managers.

Note: Docker recommends a maxium cluster size of sevent. You can go beyond seven as well, but the problem is that the performance will degrade. If you have more manager nodes, they have to also intercommunicate. So if you have a lot of manager nodes, you will have a performanc impact.

5.22 Running Manager-Only nodes in Swarm

A manager node can also run tasks Avoid this by draining the node.

5.23 Recover from loosing the quorum

Manger nodes have a internal distrubuted state store with the help of Raft

5.24 Docker System Commands

```
docker system events [OPTIONS]
```

```
docker system df # Show docker sytem info
```

Chapter 6

Orchestration using Kubernetes

Chapter 7

Installation and Configuration of Docker EE

Chapter 8

Security

8.1 Overview of Container Security Scanning

Docker containers can have security vulnerabilities.

If blindly pulled and if containers are running in production, it can result in breach.

8.1.1 Overview of Security Scanning in DTR

DTR allows us to perform security scan for the containers.

These scans can be performed ”On Push” or manually.

8.2 Configuring Container Scan with DTR

8.3 DTR Webhooks

8.3.1 Overview of the Webhooks

You can configure DTR to automatically post even notifications to a webhook URL of your choosing.

8.4 Overview of UCP Client Bundle

A client bundle is a group of certificates downloadable directly from the Docker Universal Control Plane (UCP)

Depending on the permission associate with the user, you can now execute docker swarm commands from your remote machine the take effect on the remote cluster.

- You can create a new service in UCP from your laptop
- Login to continue from your laptop without SSH (via API)

8.5 Integrating CLI with UCP Client Bundle

8.6 Overview of LDAP

Central directory where all users are stored. Depending on the federated setting users can log in.

Note: PCI-DSS standards, you need to revoke the access once they leave the organisation whether they are either resigned or terminated.

Doing in by service takes time. Can just do it in one place via LDAP.

Various solutions available which can store users centrally: - Microsoft Active Directory - RedHat Identity Management / FreeIPA

8.7 Integrating LDAP with UCP

8.8 Linux Namespaces

8.8.1 Overview of Namespaces

Docker uses a technology called namespaces to provide the isolated work space called the container.

These namespaces provide a layer of isolation. Each aspect of a container runs in a separate namespace and its access it limited to that namespace.

PID is a component of Linux namespaces. There are others like UTS and IPC. A PID basically allows you to have separation in terms of processes fo that the containers or container running here, cannot see any other processes which are running within the same operating system.

So by any change, if a specific container has been breached ther attack will not be able to connect or not be able to know the wider picture on what exactly is running.

If you are running multiple containers, each will have its own namespace in terms of process ID and network etc.

8.8.2 Namespaces in Linux

There are six different type of namespace available.

- Inter-Process Communication (IPC)
- Process (Pid)
- Network (net)
- User Id (user)
- Mount (mnt)
- UTS

8.9 Control Groups (cgroups)

8.9.1 Overview of Control Groups

Control Groups (cgroups) is a Linux kernel feature that limits, accounts for and isolates the resource usage (CPU, memory, disk I/O, network, etc) of a collection of processes.

Consider scenario where a container on a shared host is using a high amount of CPU.

Docker makes use of control gruops to control or to limit the ram and CPU that can be assigned. Now contorl gruop can not only limit by Ram and CPU, it can do a other things like disks IO etc.

However, Docker does not support each and every implementation of what cgroups suppose. The primary ones are the CPU and memory.

8.10 Limiting CPUs for Containers

There are various ways in which you can limit the CPU for containers, these include:

Approaches	
`--cpus=<value>`	If h
`--cpuset-cpus`	Limit the specific CPUs or cores a container can use. A comma-separated list or hyphen-se

```
# Commands for limited CPU
docker container run -dt --name constraint01 --cpus=1.5 busybox sh
docker container run -dt --name constraint02 --cpuset-cpus-0,1 busybox sh
```

8.11 Reservation vs Limits

By default, a container has no resource constraints and can use as much of a given resource as the host's kernel scheduler allows.

It is important not to allow a running container to consume too much of the host machine's memory.

On Linux hosts, if the kernel detects that there is not enough memory to perform important system functions, it throws an Out Of Memory exception and starts killing processes to free up memory.

8.11.1 Reservation vs Limit

Limit imposes an upper limit to the amount of memory that can potentially be used by a Docker container. Reservations on the other hand are less rigid.

When the system is running low on memory and there is contention, reservation tries to bring the container's memory consumption at or below the reservation limit.

```
# Commands for limiting RAM
docker container run -dt --name container06 -m 500m --memory-reservation 250m
    busybox sh
```

Reservation is a soft limit that the container can go beyond. Limit is a hard limit. Once the limit has been sent, the container cannot go beyond what you have set with the limit value.

8.12 Swarm MTLS

The nodes in a swarm use mutual Transport Layer Security (TLS) to authenticate, authorize and encrypt the communications with other nodes in the swarm. By default, the manager node generates a new root certificate authority (CA) along with a key pair, which are used to secure communications with other nodes that join the **swarm**

You can specify your own externally-generated root CA, using the `--external-ca` flag of the docker swarm init command.

8.12.1 Overview of Generated Certificate

Each time a new node joins the swarm, the manager issues a certificate to the node.

8.12.2 Overview of Tokens

The manager node also generates two tokens to use when you join additional nodes to the swarm: one worker token and one manager token. Each token includes the digest of the root CA's certificate and a randomly generated secret.

When a node joins the swarm, the joining node uses the digest to validate the root CA certificate from the remote manager.

8.12.3 Rotating the CA Certificate

Run `docker swarm ca --rotate` to generate a new CA certificate and key.

In the above command, docker generated a cross-signed certificate signed by the previous old CA.

After every node in the swarm has a new TLS certificate signed by the new CA, Docker forgets aboutt the old CA certificate and key material and tells all the node to trust the new CA certificate only.

Join tokens change accordingly.

8.12.4 Certificate Details

By default, each node in the swarm renews its certificate every three months.

You can configure this interval by running the `docker swarm update --cert-expiry <TIME PERIOD>`

In the event that a cluster CA key or a manager node is compromised, you can rotate the swarm root CA so that none of the nodes trust certificates signed by the old root CA anymore.

8.13 Managing Secrets in Docker Swarm

Docker secrets basically allows us to centrally manage the secret data and to also securely transmit those secrets to only the container that need to access them.

Centrally storing the secrets can be achieved with a lot of solutions. These include Hashicorp Valut.

Note: A given secret is only to those services which have been granted explicity access to it and onlly while those service tasks are running.

```
# Creating secrets
docker secret create dbcreds db_creds.txt
# Allowing containers to have access to the secret
docker service create --name webserver --secret dbcreds nginx
# Secrets location --- use docker exec -it to enter container
/run/ <--- this is a temporary file system
cd /run/secrets/ <--- should be able to see dbcreds
```

8.14 Docker Content Trust

When we are downloading an image from a network or from the internet integrity becomes true concern.

Content trust give you the ability to verify both the integrity and the publisher of all the data received from a registry over any channel.

This can be achieved with the help of digital signatures.

8.15 Overview of Docker Groups

`ps aux | grep docker` Docker runs as a root user. So everything will work.

Do not add to sudoers as this will give the user the ability to run other commands.

Docker recommends creating a docker group. See /etc/group. Should see docker. Add user name there. `su - newuser` and then run `docker ps`. This user "newuser" should be able to see all the container. This user does not have sudo privilleges. Can also use `usermod -aG docker dockeruser`

It is not a best practice to directly modify the group file. Use the above command instead.

8.16 Overview of Linux Capabilities for Docker

There are several types of capabilities which Linux provides to have granular access at the application level.

Capability	Action
SETCAP	Yes
CHOWN	Yes
	No
	No

Depending upon the capabilities which are assigned or not assigned, the container can or cannot do a certain type of operation. Even if you are logged in as root. These capabilities can be added.

This can be very useful when you want to have granular control over what a specific container can do. It can be useful during the hardening of your process.

8.17 Privileged Containers

By default, a docker container does not have many capabilities assigned to it. Docker containers are also not allowed to access any devices. Hence by default, a docker container cannot perform various use-cases like ”run docker container” inside a docker container.

8.17.1 Privileged Containers

Privileged Containers can acess all the devices on the host as well as have configuration in AppArmore or SELinux to allow the contaienr nearly all the same access to the host as processes running outside containers on the host. - Limits that you set for privileged containers will not be followed. - Running a prilvileged flag gives all the capabilities to the container.

```
# Start a prilvileged container
docker run -dt --privileged amazonlinux bash
```

All devices available to the host are also available to this container. In a privileged container, Linux capabilities are not restricted.

Since these have access to the host, their usage should be restricted.

Chapter 9

Storage and Volumes